

# Mellow Language Reference

*Core syntax, keywords, and built-in functions*

Mellow Language project

## 1. 1. Overview

Mellow is an English-influenced programming language implemented in C. Function calls use angle brackets, code blocks use square brackets, and curly braces are used for array data. Arithmetic and comparison are expressed with named built-in functions rather than operators.

This reference describes the current executable core. Keywords marked "reserved" are recognized by the lexer but are not yet executable in the interpreter.

## 2. 2. Basic Syntax

Function calls:

```
print<"Hello, Mellow!">
add<2, 3>
print<add<2, mul<3, 4>>>
```

Blocks:

```
if gt<score, 10> [
  print<"high score">
]
```

Comments use `//` for one line and `/* ... */` for multiple lines. Newlines may be used between arguments and delimiters in calls and collection literals.

**3. 3. Values and Variables** **null** The absence of a value. The literal is `.B null`. **number** The current runtime stores integer and floating-point literals as C double values. Examples: `.B 42` and `.B 3.14159`. **string** Text enclosed in double quotes. Example: `.B "hello"`. **char** A single character enclosed in single quotes. Character values currently use the same string representation at runtime. **bool** The boolean literals `.B true` and `.B false`. **array** An ordered collection written with curly braces. Example: `.B {1, 2, 3}`. **list** An ordered dynamic collection created with `.B list<...>`. Example: `list<"red", "green", "blue">`.

**let** defines a mutable variable:

```
let name = "Mellow"
set<name, "Mellow Language">
```

**const** defines a variable that cannot be reassigned or passed as the first argument to `.B set`:

```
const PI = 3.14159
```

**4. 4. Implemented Keywords** **true** Creates the boolean value true. **false** Creates the boolean value false. **null** Creates the null value. **let** Declares a mutable variable using `.B let name = expression`. **const** Declares an immutable variable using `.B const name = expression`. **if** Executes a block when an expression is truthy. Truthy values are true, nonzero numbers, nonempty strings, and nonempty collections.

```
if eq<name, "Mellow"> [
  print<"matched">
]
```

**else** Provides the alternative block for `.B if`. Chained `.B else if` syntax is supported. **while** Repeats a block while its

condition is truthy.

```
while gt<count, 0> [  
  print<count>  
  set<count, dec<count>>  
]
```

**loop** Repeats a block indefinitely until .B break or a runtime error occurs. **break** Leaves the nearest loop. **continue** Skips to the next iteration of the nearest loop. **return** Returns a value from a named function. With no expression it returns null. **func** Defines a named function with parameters and a square-bracket body.

```
func square<x> [  
  return mul<x, x>  
]  
print<square<5>>
```

**raise** Raises a runtime exception and exits the current program with a nonzero status. The usual form is .B raise<"message">.

## 5. 5. Arithmetic Builtins

Arithmetic operators such as +, -, \*, /, and % do not exist in Mellow. **add<a,b>** Returns the numeric sum. It also concatenates two strings or two collections of the same kind. **sub<a,b>** Returns a minus b for numbers. **mul<a,b>** Returns a times b for numbers. **div<a,b>** Returns a divided by b. Division by zero is a runtime error. **mod<a,b>** Returns the floating-point remainder of a divided by b. **pow<a,b>** Returns a raised to the power b. **sqrt<a>** Returns the square root of a. **inc<a>** Returns a plus one. **dec<a>** Returns a minus one.

## 6. 6. Comparison and Logic Builtins

Comparison operators such as ==, !=, <, >, <=, and >= do not exist as operators. The following functions return booleans. **eq<a,b>** True when the values are equal, including recursive collection equality. **neq<a,b>** True when the values are not equal. **lt<a,b>** True when numeric a is less than numeric b. **gt<a,b>** True when numeric a is greater than numeric b. **lte<a,b>** True when numeric a is less than or equal to numeric b. **gte<a,b>** True when numeric a is greater than or equal to numeric b. **equal<a,b>** Readable alias for .B eq<a, b>. **not\_equal<a,b>** Readable alias for .B neq<a, b>. **less<a,b>** Readable alias for .B lt<a, b>. **greater<a,b>** Readable alias for .B gt<a, b>. **less\_equal<a,b>** Readable alias for .B lte<a, b>. **greater\_equal<a,b>** Readable alias for .B gte<a, b>. **and<a,b>** Returns the boolean conjunction of a and b. **or<a,b>** Returns the boolean disjunction of a and b. **not<a>** Returns the boolean negation of a.

**7. 7. Collection and Conversion Builtins** **list<items>** Creates a list from zero or more values. **len<value>** Returns the character count of a string or the item count of an array/list. **type<value>** Returns a string naming the runtime type: null, number, string, bool, array, or list. **to\_string<value>** Converts primitive values to text. Collections currently return a collection placeholder string and are displayed directly by .B print. **print<values>** Prints each argument separated by a space, followed by a newline. **set<name,value>** Replaces the value of a mutable variable. The first argument is a variable name, not an evaluated expression.

## 8. 8. Interpolation

Expressions inside a string using {name} are replaced with the current value of the named variable:

```
let user = "Ada"  
print<"Hello, {user}!">
```

Interpolation currently supports variables containing strings, numbers, and booleans. Unknown names remain unchanged.

## 9. 9. Functions and Recursion

Named functions support parameters, local variables, return values, and recursion. Arguments are evaluated before the function is called.

```
func factorial<n> [  
  if lte<n, 1> [  
    return 1  
  ]  
  return mul<n, factorial<dec<n>>>  
]  
print<factorial<5>>
```

**10. 10. Command-Line Tools make Builds the interpreter. Run it from the project root. ./mellowfile.mll Executes a Mellow source file. file.mll./mellow--tokens Prints the lexer token stream with source locations. ./mellow--repl Starts the interactive read-evaluate-print loop. Type .B exit to leave it.**

## **11. 11. Reserved Keywords**

The following words are lexed and reserved for the next runtime layers: **for**, **class**, **public**, **private**, **import**, **try**, **catch**, **fn**, **this**, **super**, and.

**for** is reserved for collection iteration. .B class, .B public, and .B private are reserved for the object system. .B this and .B super are reserved for method access. .B import is reserved for modules. .B try and .B catch are reserved for structured error recovery. .B fn is reserved for anonymous functions and closures. These features are specified in the project roadmap but are not executable in the current core.

## **12. 12. Current Limitations**

Dictionaries, indexing, default parameters, lambdas, closures, structured try/catch recovery, modules, classes, inheritance, and garbage collection are not yet implemented. The lexer recognizes much of their vocabulary so these features can be added without changing the central call and block syntax.

